

PRACTICAL NO 4

Aim: Write a program to implement a Doubly Linked List.

INPUT:

```
class DoublyLinkedList:
    def __init__(self, data):
        self.data = data
        self.tail = None
        self.next = None
        self.head = None

    def insertion_at_beginning(self, data):
        newNode = DoublyLinkedList(data)
        if self.head is None:
            self.head = newNode
        else:
            newNode.next = self.head
            self.head.tail = newNode
            self.head = newNode

    def insertion_at_end(self, data):
        newNode = DoublyLinkedList(data)
        current = self.head

        while current.next is not None:
            current = current.next
        current.next = newNode
        newNode.tail = current

    def insertion_at_position(self, data, position):
        newNode = DoublyLinkedList(data)
        if position == 1:
            newNode.next = self.head
            self.head.tail = newNode
            self.head = newNode
        else:
            current = self.head
            current_position = 1
            while current_position < position - 1 and current.next is not None:
                current = current.next
                current_position += 1
            if current.next is None:
                current.next = newNode
                newNode.tail = current
            else:
                newNode.next = current.next
                newNode.tail = current
                current.next.tail = newNode
                current.next = newNode
```

```

def deletion_at_beginning(self):
    self.head = self.head.next
    if self.head is not None:
        self.head.tail = None

def deletion_at_end(self):
    current = self.head

    while current.next is not None:
        current = current.next
    current.tail.next = None

def deletion_at_position(self, position):
    if self.head is None:
        return
    elif position == 1:

        self.head = self.head.next
        if self.head is not None:
            self.head.tail = None
    else:
        current = self.head
        current_position = 1

        while current_position < position - 1 and current.next is not None:
            current = current.next
            current_position += 1
        if current.next is None:
            return
        else:
            current.next = current.next.next
            if current.next is not None:
                current.next.tail = current

def display(self):
    newNode = self.head
    while newNode is not None:
        print(newNode.data, end=" ")
        newNode = newNode.next
    print()

```

```
dll = DoublyLinkedList(10)

dll.insertion_at_beginning(20)
dll.insertion_at_beginning(30)
dll.insertion_at_beginning(40)

dll.insertion_at_end(50)
dll.insertion_at_end(70)
dll.insertion_at_end(80)

dll.insertion_at_position(60, 3)

print("Original List:")
dll.display()

dll.deletion_at_beginning()
dll.deletion_at_end()
dll.deletion_at_position(3)

print("Modified List:")
dll.display()
```

OUTPUT:

```
Original List:
40 30 60 20 50 70 80
Modified List:
30 60 50 70
PS D:\Coding\college-pr\DSA_I
```